Dynamics Specification for Neural Control Lyapunov Function and Neural Controller Training

Gil Benezer
Masters of Science in Artificial Intelligence
Khoury College of Computer Science
Northeastern University
Boston, MA
benezer.gi@northeastern.edu

Fang-Hsin Li
Masters of Engineering
Electrical and Computer Engineering, College of Engineering
Northeastern University
Boston, MA
li.fangh@northeastern.edu

Abstract

Neural network-based control Lyapunov functions offer a promising approach for certifying stability in closed-loop control systems, with recent advances enabling scalable verification through over-approximation methods like α, β-CROWN. However, an existing implementation for synthesis and verification relies on hard-coded dynamical system specifications, limiting its applicability to novel systems. This project worked towards resolving this issue by developing a flexible framework for defining continuous-time nonlinear dynamical systems using SymPy and discretizing them in a manner compatible with a state-of-the-art neural Lyapunov training algorithm. To support qualitative and quantitative evaluation, novel visualization and numerical integration tools for analyzing Lyapunov functions and regions-of-attraction were developed. This framework was validated against hard-coded implementations on a partially-observable inverted pendulum system and a fully observable path-tracking system, demonstrating empirical comparability across 25 independent training runs for each. Results show that while the framework successfully enables system specification and training, convergence seems to remain sensitive to hyperparameter selection. This project presents a meaningful step towards a general-purpose tool for control Lyapunov function synthesis and verification. The project GitHub repository can be found at `https://github.com/gbenezer/Lyapunov_Stable_NN_Controllers_Custom_Dynamics`

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

1 Introduction

The advent of novel machine learning approaches for constructing neural-network based controllers during this artificial intelligence boom is a boon for a variety of industries. However, most techniques for controller synthesis do not yield safety guarantees which are important for highly regulated, high stakes applications. However, most techniques for controller synthesis still lack safety guarantees, which are critical for highly regulated or high-stakes applications. A key safety-related property in control theory is stability, which informally requires that the closed-loop system (the dynamical process together with its controller) does not behave uncontrollably. One notion of stability is Lyapunov stability (1; 2): if a closed loop system starts near a stable equilibrium, it will at least stay within a bounded distance from that equilibrium in the future, with stronger notions of stability requiring exact return to the equilibrium with explicit rates of convergence.

A standard method for certifying this type of stability for a closed-loop system is to construct a control Lyapunov function (CLF). A CLF is a scalar function that is zero at the equilibrium, strictly positive elsewhere in the feasible state space, and guaranteed to decrease along trajectories starting within a region-of-attraction (ROA). Existence of such a function certifies stability for all states in the ROA. It is not straightforward to construct such a function, though these certificate functions have been synthesized for simple dynamical systems and controllers (2). Modern approaches use neural networks as Lyapunov functions and then verify these functions indeed satisfy the Lyapunov conditions for a particular ROA (2; 3; 4). In particular, the authors in (4) improve upon the methods in (3) by using over-approximation methods (5; 6; 7; 8; 9; 10) instead of exact verifiers to help scale to larger neural networks. Beyond that improvement, a novel formulation of the Lyapunov conditions for defining ROA implicitly in addition to extension of neural network Lyapunov functions to output-feedback control (where the state is not fully known and has to be estimated) mean that this is a promising tool for assuring the safety of control systems.

However, the code submitted for the paper itself is not yet ready for general use (11). While the `README.md` for the project lists how to run some example scripts for neural network Lyapunov candidate generation, they use hard-coded dynamical system classes for some (but not all) of the toy systems the authors test in the paper. They do provide a base class for the definition of general model dynamics in `ROOT/neural_lyapunov_training/models.py`, but there are no examples of how to use them for training Lyapunov functions or associated controllers. To move towards improving the usability of the approaches in (4) on more general nonlinear systems, we developed a framework for defining continuous-time dynamical systems using SymPy (12) and a wrapper class to discretize those systems that is compatible with those example scripts assuming those represent the intended usage of the code base. Additionally, new functionality for visualization of Lyapunov functions and numerical integration of regions-of-attraction to compare relative volumes using Plotly (13) and Scipy (14) have been added, respectively.

2 Related Work

2.1 Safe / Constrained RL + HJI Reachability

Methods such as Safe Reinforcement Learning and Constrained RL are among the most widely explored learning-based strategies for enforcing safety in control systems. These approaches typically incorporate safety through constraint penalties, Lagrangian relaxations, shielding mechanisms, or risk-sensitive objectives, and their key advantage is the ability to handle unknown or uncertain dynamics by learning directly from interaction (2). However, a fundamental limitation of Safe RL is that safety is rarely provably guaranteed: even when constraints are included in the reward or policy-update process, the resulting controller usually does not provide a formal certificate ensuring safety or stability for all possible trajectories (2).

In contrast, Hamilton–Jacobi (HJ) and Hamilton–Jacobi–Isaacs (HJI) reachability methods offer strong, formal guarantees by computing value functions whose sub-level sets characterize robustly invariant safe sets or reach-avoid sets under worst-case disturbances. While this yields rigorous certificates of safety, solving the associated partial differential equations requires grid-based methods that scale poorly with the dimension of the state space (2), making HJ/HJI reachability difficult to apply to higher-dimensional or real-time systems.

This trade-off between empirical flexibility and formal guarantees motivates Lyapunov-based approaches. Once a suitable control Lyapunov function (or control barrier function) is constructed, it acts as a certificate of stability or safety for the closed-loop system, while avoiding the full PDE solve required by HJ/HJI methods (2).

In this sense, Lyapunov-based methods aim to retain the rigor of formal certificates with improved scalability, especially when combined with expressive function approximation methods such as neural networks, an approach that has recently gained significant traction (3), (4).

2.2 Classical and Modern Methods for Constructing Lyapunov Functions

2.2.1 Traditional Lyapunov synthesis methods

To address scalability limitations while still providing formal guarantees, Lyapunov based methods offer an alternative framework. A Lyapunov function certifies stability by ensuring that the system state decreases toward an equilibrium under the closed-loop dynamics (2; 15). Classical methods to construct Lyapunov functions include quadratic Lyapunov functions for linear systems, polynomial functions solved through sum-of-squares (SOS) optimization for polynomial systems, and piecewise or hybrid Lyapunov functions (2; 16). These approaches can provide provable stability or safety guarantees but are often limited by system structure or lack of scalability in higher dimensions. (2)

2.2.2 Neural Lyapunov Functions

To overcome these constraints, recent work has explored using neural networks to parameterize Lyapunov functions and associated stabilizing controllers. Neural Lyapunov methods aim to preserve the formal structure of Lyapunov certificates while leveraging function approximation for general nonlinear systems. (3; 17; 18). Prior work has demonstrated the feasibility of learning Lyapunov functions or control Lyapunov functions for continuous-time and discrete-time systems, though most methods still rely on hand-specified dynamics or limited model classes. (3) These limitations motivate the direction taken in our work, which aims to develop a flexible framework that enables Lyapunov-based learning for a broader class of nonlinear dynamical systems; similar motivations are echoed in survey discussions on scalability and generality (2), and in recent formulations for certifiable neural controllers. (4)

Prior work on constructing Lyapunov functions has largely focused on continuous-time systems, typically under structural assumptions such as linear or polynomial dynamics, which make analytical techniques or SOS optimization applicable (17; 18; 19). More recent methods parameterize Lyapunov functions with neural networks, but often restrict the control policy to be linear and remain tailored to continuous-time settings, limiting their effectiveness for discrete-time systems(17; 18). For discrete-time hybrid systems, convex Lyapunov functions have been learned via MILP formulations, although these approaches apply only to piecewise-affine dynamics and do not synthesize controllers (20). Other works impose restrictive neural network structures, such as requiring ReLU Lyapunov functions or quadratic forms, or provide only empirical rather than formal guarantees (3; 2).

In parallel, deep learning has enabled high-performance neural controllers for robotics (21), yet these policies generally lack formal stability guarantees (4). Classical Lyapunov theory provides such guarantees, and tools like LQR and SOS programming have been used to certify regions of attraction for linear or polynomial controllers (4). Extending these guarantees to complex neural controllers remains challenging (4).

2.2.3 Current Trends in Lyapunov Function Construction

Recent research follows two main directions:

- Data-driven approaches that search for Lyapunov functions or controllers using sampled trajectories (22; 2; 23; 24)
- Verification-based approaches that use SMT, MIP, or SDP tools to certify Lyapunov conditions (17; 19)

Verification-based methods offer stronger guarantees, but SMT solvers struggle with nonlinear neural dynamics and MIP techniques require piecewise-linear approximations, which limits their applicability (4),. Consequently, neural Lyapunov methods remain conservative and tend to certify only small regions of attraction (4).

Advances in neural verification such as α-CROWN and β-CROWN provide scalable linear bound-propagation tools that can certify large neural networks and closed-loop systems (5; 8; 10). Building on these developments, recent work aims to jointly synthesize neural controllers and Lyapunov functions with formal guarantees for both state and output feedback (4). These trends highlight the need for a more flexible and general framework for Lyapunov-based learning in nonlinear systems, which motivates the approach taken in this work.

3 Mathematical Background

As we are expanding upon the work in (4), we will use and re-introduce their framework for dynamical systems and stability. Specifically, we will consider discrete-time dynamical systems $x_{t+1} = f(x_t, u_t)$ with observation dynamics $y_t = h(x_t)$. $x_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ is the state of the system at time t within the set of possible states $\mathcal{X}$, $y_t \in \mathbb{R}^{n_y}$ is the observation of the system at time t, and $u_t \in \{u | u \in [u_{\min}, u_{\max}]\} \subset \mathbb{R}^{n_u}$ is the control action at time t selected from the set of admissible control actions. The goal state and goal control actions at an equilibrium point of the system are denoted as x^* and u^*

In the simplest case, the dynamical system is fully observable, meaning that $y_t = h(x_t) = x_t$ and the exact state of the dynamical system is known with perfect precision for all time t. Then the appropriate feasible control action u_t can be defined by a neural network policy $\phi_\pi(x_t) : \mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$ as

$$\pi(x_t) = u_t = \text{clamp}(\phi_\pi(x_t) - \phi_\pi(x^*) + u^*, u_{\min}, u_{\max})$$

However, when the dynamical system is not fully observable (i.e. $y_t \neq x_t$), the actual state of the system needs to be estimated. Within the framework of (4), this estimation is done by a neural network observer $\phi_{\text{obs}}(\hat{x}_t, y_t) : \mathbb{R}^{n_x} \times \mathbb{R}^{n_y} \to \mathbb{R}^{n_x}$ such that

$$\hat{x}_{t+1} = f(\hat{x}_t, u_t) + \phi_{\text{obs}}(\hat{x}_t, y_t - h(\hat{x}_t)) - \phi_{\text{obs}}(\hat{x}_t, \overrightarrow{\mathbf{0}}_{n_y})$$

and the neural network controller now is a mapping $\phi_\pi^{\text{obs}}(\hat{x}_t, y_t) : \mathbb{R}^{n_x} \times \mathbb{R}^{n_y} \to \mathbb{R}^{n_u}$ that takes in state estimates and output observations and yields control action

$$\pi^{\text{obs}}(\hat{x}_t, y_t) = u_t = \text{clamp}(\phi_\pi^{\text{obs}}(\hat{x}_t, y_t) - \phi_\pi^{\text{obs}}(x^*, h(x^*)) + u^*, u_{\min}, u_{\max})$$

To talk about these two separate cases at the same time, the authors of (4) introduce the concept of internal state ξ_t: in the case of state feedback, $\xi_t \equiv x_t$, but for output feedback the internal state is augmented with the state prediction error ($\xi_t \equiv [x_t, \hat{x}_t - x_t]^T = [x_t, e_t]^T$).

The assumed dynamics discussed so far have been open-loop in nature, not considering the impact of the controller itself. For state feedback, the closed loop dynamics in terms of the internal state is just $f_{cl}(\xi_t) = f(x_t, \pi(x_t))$. For output feedback, the closed loop dynamics of the internal state (true state and state estimate error) are

$$f_{cl}(\xi_t) = \begin{bmatrix} f(x_t, \pi^{\text{obs}}(\hat{x}_t, h(x_t)) \\ f(\hat{x}_t, \pi^{\text{obs}}(\hat{x}_t, h(x_t))) + g(x_t, \hat{x}_t) - x_t \end{bmatrix}$$

where

$$g(x_t, \hat{x}_t) \equiv \phi_{\text{obs}}(\hat{x}_t, y_t - h(\hat{x}_t)) - \phi_{\text{obs}}(\hat{x}_t, \overrightarrow{\mathbf{0}}_{n_y})$$

is the difference in observer network output due to errors in state estimation.

The working definition of stability for the closed-loop dynamical system for (4) that will be adopted here is exponential stability. Assuming the system starts in some initial state $x_0 \in \mathcal{X}_0 \subseteq \mathcal{X} \subseteq \mathbb{R}^{n_x}$, we define the trajectory map $\tau_\pi(x_0, t) : \mathcal{X}_0 \times \mathbb{N} \to \mathcal{X}$ as the set of states that trajectories of the closed loop dynamical system can reach under control policy π starting from state x_0. Exponential stability means that there exist some positive constants $C, \lambda > 0$ such that

$$||\tau_\pi(x_0, t) - x^*|| \leq C \cdot ||x_0 - x^*|| \cdot e^{-\lambda \cdot t} \quad \forall x_0 \in \mathcal{X}_0$$

This condition means that all trajectories of the closed-loop system that start in the set $\mathcal{X}_0$ will converge to the equilibrium state x^* exponentially fast with a rate of convergence λ. The largest such forward invariant set $\mathcal{X}_0$ of a specific equilibrium state ξ^* is denoted as $\mathcal{R}$ and called its region-of-attraction (ROA).

The main goal of the algorithms developed in (4) is to certify a given closed-loop dynamical system is exponentially stable around a particular equilibrium point and to find an inner approximation $\mathcal{S} \subseteq \mathcal{R}$ to the ROA by constructing a neural network Lyapunov function. A Lyapunov function is a scalar function $V(\xi_t) : \mathbb{R}^{n_\xi} \to \mathbb{R}$ such that the following conditions hold:

$$V(\xi_t) > 0 \quad \forall \xi_t \in \mathcal{S} \neq \xi^* \text{ and } V(\xi_t) = 0 \text{ when } \xi_t = \xi^*$$

$$V(\xi_{t+1}) - V(\xi_t) \leq -\kappa \cdot V(\xi_t) \quad \forall \xi_t \in \mathcal{S}$$

The algorithms also aim to maximize the volume of the inner approximation $\text{Vol}(\mathcal{S})$, subject to the above constraints. In (4) and this work, we focus on the following parameterizations of the Lyapunov function as a neural network

$$V(\xi_t) = |\phi_v(\xi_t) - \phi_v(\xi^*)| + ||(\epsilon I + R^T R) \cdot (\xi_t - \xi^*)||_1$$

or quadratic function

$$V(\xi_t) = (\xi_t - \xi^*)^T \cdot (\epsilon I + R^T R) \cdot (\xi_t - \xi^*)$$

where ϵ is a small positive scalar and $R \in \mathbb{R}^{n_\xi \times n_\xi}$ is a matrix of trainable entries to be optimized, possibly along with neural network weights.

In practice, neural network verification has to be conducted over an explicitly defined and often axis-aligned bounding hyper-rectangle $\mathcal{B} \equiv \{\xi_t | \xi_r \in [\xi_{\min}, \xi_{\max}]\}$. However, restricting consideration to such a bounding box often reduces the possible size of $\mathcal{S}$ because most approaches look for an inner approximation that is entirely contained within $\mathcal{B}$. The authors of (4) re-formulate the Lyapunov constraints in such a way that the inner approximation set $\mathcal{S}$ can be defined as the intersection of $\mathcal{B}$ and a sub-level set of the Lyapunov function

$$\mathcal{S} \equiv \{\xi_t \in \mathcal{B} | V(\xi_t) < \rho\}$$

for an appropriately chosen threshold ρ. Letting

$$F(\xi_t) \equiv V(f_{cl}(\xi_t)) - (1 - \kappa) \cdot V(\xi_t)$$

be a function related to the second Lyapunov constraint and

$$H(\xi_{t+1}) \equiv ||\text{ReLU}(\xi_{t+1} - \xi_{\max})||_1 + ||\text{ReLU}(\xi_{\min} - \xi_{t+1})||_1$$

be a function that measures the degree to which ξ_{t+1} fails to stay within bounding hyper-rectangle $\mathcal{B}$, then the authors show that maximizing the volume of $\mathcal{S}$ subject to the Lyapunov constraints is the same as the following optimization problem

$$\min_{V, \pi, \phi_{\text{obs}}} L_{\text{ROA}}(\rho) = \sum_i^{n_{\text{candidate}}} \text{ReLU}\left(\frac{V(\xi_{\text{candidate}}^{(i)})}{\rho} - 1\right)$$

subject to

$$\min([\text{ReLU}(F(\xi_t)) + c_0 \cdot H(\xi_{t+1})], [\rho - V(\xi_t)]) \leq 0 \quad \forall \xi_t \in \mathcal{B}$$

for an appropriately chosen set of candidate internal states $\xi_{\text{candidate}}$ and sub-level set threshold ρ, with c_0 being a hyperparameter to weight how heavily the optimizer should weight bounding hyper-rectangle violations relative to Lyapunov derivative constraint violations.

4 Methods

To step back from the Lyapunov optimization and focus on the crux of our extension, we need to discuss the approach that they use for dynamical system definition. Specifically, in their `README.md` file they explicitly guide users to examples of using their framework for controller, observer, and Lyapunov function synthesis. Choosing between dynamical systems (according to the differences between example files) entails switching out explicitly-defined, hard-coded continuous time nonlinear dynamical system Python classes; these classes are not standardized with respect to how they function and what fields or methods are defined for them. One of these continuous time classes is then wrapped in a wrapper class that handles discretization of the continuous time dynamics, with different classes for first-order and second-order systems.

To make it easier for users to construct nonlinear dynamical systems that are compatible with this Lyapunov function framework, we define a generic `SymbolicDynamicalSystem` superclass and a wrapper `GenericDiscreteTimeSystem` class that are both PyTorch Modules. As a concrete usage example, we define a partially-observable nonlinear inverted pendulum system similar in form to that in (3) and delineate both the state space formulation and corresponding subclass code in Appendix section A.1.1.

To define the system, the user subclasses SymbolicDynamicalSystem and writes an `__init__` method and a `define_system` method (which is called during initialization). The `define_system` method requires the user to use SymPy (12) to define symbolic variables for state variables, control variables, and parameter variables. Once those are constructed (and associated with numeric values in the case of parameters), the user then symbolically defines the true nonlinear dynamics with the `self._f_sym` private field and the private observation function `self._h_sym`. If needed, the user can also override the system's equilibrium state and control x^* and u^* which are zero vectors by default.

Once a system is defined and instantiated in this way, the superclass has methods to

- Evaluate the nonlinear dynamics and observations at a given point in state space
- Symbolically derive and numerically evaluate linearizations of dynamics and observations around an equilibrium point
- Check whether the system's equilibrium point is stable
- Check if a particular state/control action pair is numerically stable
- Calculate LQR and Kalman gains for the linearized system around its equilibrium

The defined system can also be passed to the `GenericDiscreteTimeSystem` wrapper class, along with a discretization time scale and an integration method to enable

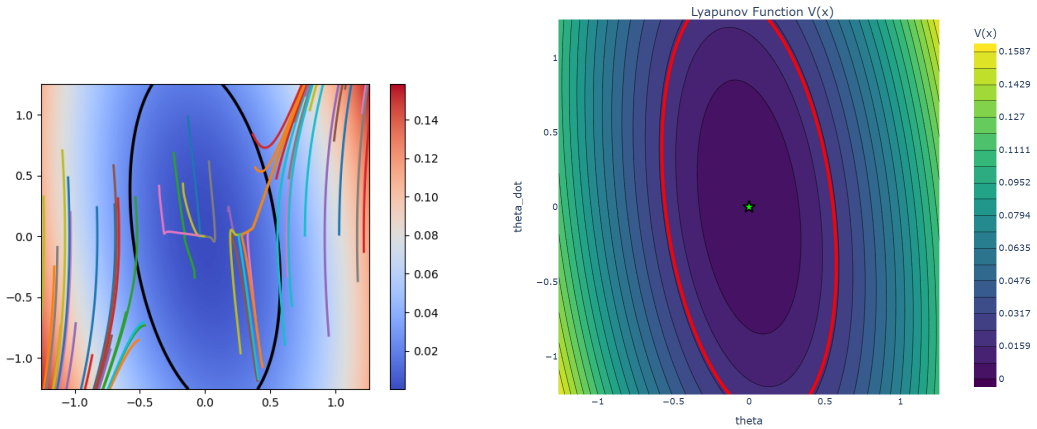
- Integrate an arbitrary-order system with explicit Euler, midpoint, or fourth-order Runge-Kutta methods to simulate trajectories
- Calculate discrete-time LQR and Kalman gains for the linearized system around its equilibrium
- Plot sets of trajectories in phase space
- **Directly interface with the code base developed for** (4)

We also wanted to have a more generalizable method to plot Lyapunov functions and their associated controllers and observers for qualitative comparison. As such, we used Plotly (13) to make 2D and 3D interactive HTML plots for visualizing both the behavior of the Lyapunov function on its own (similar to the visualizations of the authors) in addition to the discrete-time Lyapunov derivative $\Delta V = V(\xi_{t+1}) - V(\xi_t)$ with a given controller and observer to measure the degree to which the derivative condition is actually enforced over the region of interest $\mathcal{B}$. The main limitation for our implementation of ΔV visualization and quantitation (described below) is that our code assumes the observer has converged to output the true state, meaning that the state estimation error $e_t = \hat{x}_t - x_t \approx 0$. It would take significant refactoring of the code to change this, so we leave it as an exercise for future researchers.

Finally, to have an explicitly quantitative method to measure and compare ROAs, we created a framework to carry out ROA numerical integration. We initially implemented both a Monte-Carlo integration method that randomly samples states $\xi_{\text{sample}} \in \mathcal{B}$ and a uniform grid-based integration method that sub-divides $\mathcal{B}$ into cells and selects cell centers ξ_{grid}. Given that the ROA inner approximation $\mathcal{S}$ is defined in terms of the intersection of $\mathcal{B}$ and a sub-level set threshold ρ, the ROA area can be estimated as either the volume of $\mathcal{B}$ multiplied by the fraction of samples $V(\xi_{\text{sample}}) \leq \rho$ (MC method) or the number of grid cells with centers $V(\xi_{\text{grid}}) \leq \rho$ multiplied by the cell volume (grid). Inspired by some Bayesian optimization methods for approximating expected values for acquisition functions with quasi-Monte Carlo and facing a need for increased sample efficiency in higher dimensions, we also implemented quasi-Monte Carlo methods that use low discrepancy sequences (Sobol and Halton sequences) to generate random sample states $\xi_{\text{sample}}^{(ld)}$ that more uniformly cover hyper-volume $\mathcal{B}$. Finally, after identifying some interesting trends in the visualizations of the Lyapunov derivatives, we also moved forward to apply the integration methods to find the portion of the hyper-volume where $\Delta V < 0$

5 Results

We first wanted to make sure that the default visualizations that their example scripts generate are qualitatively the same as the visualizations that we generate. To continue with the concrete example of the partially observable inverted pendulum system, we first compared the visualizations of the default output for the system and our visualization.



(a) Default 2D $V(\xi)$ Visualization

(b) Our 2D $V(\xi)$ Visualization

Figure 1: 2D Lyapunov Function Visualization Comparison, Pendulum Output Feedback

As shown in Figure 1, both visualizations qualitatively show the same phenomenon. The main difference is that we do not by default visualize a set of predefined trajectories (though our visualization allows one to do this). We have tested this with all the other example scripts and the visualizations align similarly.

The next step for verifying that our extensions are valid is to test the behavior of the symbolic system we created against the hard-coded system that the authors use.

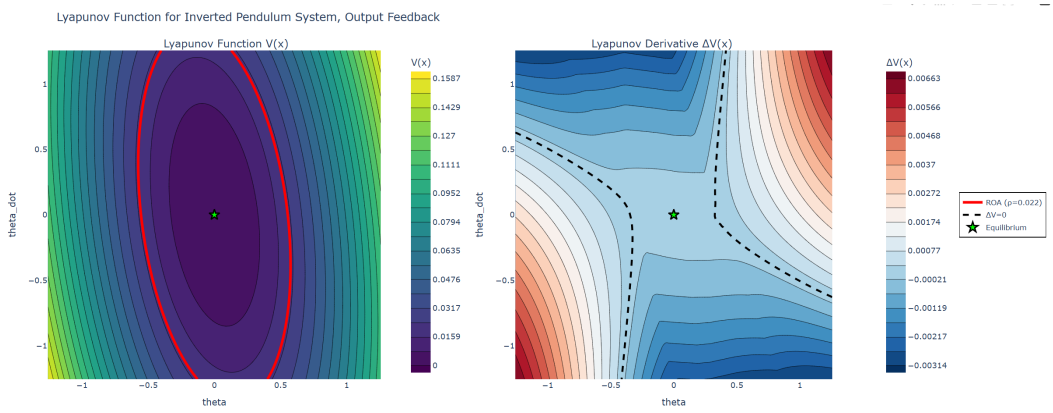


Figure 2: Full 2D Visualization, Hard-Coded

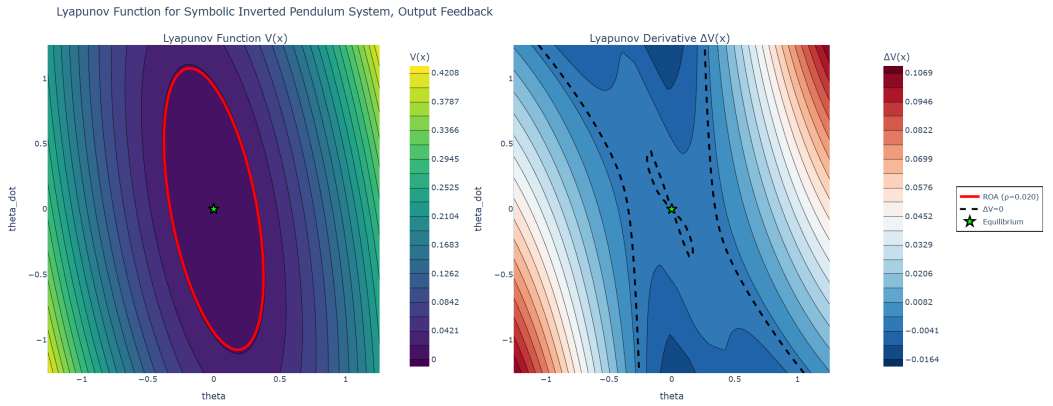


Figure 3: Full 2D Visualization, Symbolic

There is a clear difference in the size of the ROA inner approximation $\mathcal{S}$ as well as the contours of the Lyapunov derivative ΔV in Figures 2 and 3, . One interesting thing to note is that there are regions of $\mathcal{S}$ where $V(\xi) \leq \rho$ but also $\Delta V(\xi) > 0$ for both hard-coded and symbolic systems, showing that while the constructed Lyapunov candidate could be an open-loop Lyapunov function, it is not a control Lyapunov function when coupled with the trained controller $\pi(\hat{x}_t, y_t)$ over $\mathcal{S}$.

These violations are at least in part due to the formulation of the training. The authors use the Counter-Example Guided Inductive Synthesis (CEGIS) framework (25) where the optimization problem presented at the end of the Background section is treated as a bi-level optimization problem. The inner optimization finds adversarial internal states ξ_{adv} by maximizing a loss derived from the constraint condition

$$L_{\dot{V}}(\xi_t; \rho) \equiv \text{ReLU}(\min([\text{ReLU}(F(\xi_t)) + c_0 \cdot H(\xi_{t+1})], [\rho - V(\xi_t)]))$$

and constructs a counterexample dataset $\mathcal{D}$ for the outer optimization of Lyapunov function, controller, and observer parameters (all lumped in θ). The overall loss function for the outer optimization is

$$L(\theta; \mathcal{D}, \rho) = \sum_{\xi_{\text{adv}}^{(i)} \in \mathcal{D}} L_{\dot{V}}(\xi_{\text{adv}}^{(i)}; \rho) + c_1 \cdot L_{\text{ROA}}(\rho) + c_2 \cdot ||\theta||_1 + c_3 \cdot L_{\text{obs}}$$

$$L_{\text{obs}} \equiv \sum_{\xi_t \in \mathcal{C}} ||\hat{x}_{t+1} - x_t||_1$$

where c_1, c_2, c_3 are positive scalar constants and $\mathcal{C}$ is a set of randomly chosen internal states ξ in hyper-volume $\mathcal{B}$.

Each of the loss terms (derivative violation, ROA volume, parameter sparsity, and observer) may or may not be in tension. Only if the derivative loss term $\sum_{\xi_{\text{adv}}^{(i)} \in \mathcal{D}} L_{\dot{V}}(\xi_{\text{adv}}^{(i)}; \rho)$ is exactly zero over the entire inner approximation $\mathcal{S}$ is the Lyapunov function a control Lyapunov function under the closed-loop dynamics for the synthesized controller and observer. If that loss term is not zero, then there are adversarial states on or within the boundary of $\mathcal{S}$ where the Lyapunov derivative is greater than zero.

To measure the performance of each approach according to the volume of $\mathcal{S}$, the volume of $\mathcal{B}$ over which $\Delta V \leq 0$, and the intersection of the two regions, we carried out numerical integration as described in our Methods section.

Metric	Author Hard-Coded System	Our Symbolic System
ρ	0.022181 ± 0.007968	0.023006 ± 0.003195
Fraction of $\mathcal{B}$ with $V(\xi) \leq \rho$	0.1781 ± 0.0714	0.174 ± 0.0477
Fraction of $\mathcal{B}$ with $\Delta V(\xi) \leq 0$	0.4977 ± 0.0554	0.4237 ± 0.0672
Fraction of $\mathcal{B}$ in Intersection	0.1781 ± 0.072	0.1736 ± 0.0522
Fraction of $\mathcal{S}$ where $\Delta V(\xi) > 0$	0.0131 ± 0.0131	0.001798 ± 0.001798

Table 1: Pendulum Output Feedback Metrics, Median $\pm$ Median Absolute Deviation, $N = 25$

As demonstrated by the metrics in Table 1, collected over 25 independent runs, the two system types do not behave in practically different ways. We therefore conclude that, at least for the pendulum output system, the two approaches behave equivalently in a statistical sense and can be used interchangeably.

We initially tried to carry out these experiments for the inverted pendulum system with full state information. However, both the hard-coded system and the symbolic system did not find meaningful ROA approximations with the default example file configuration. The authors knew this, and so included a particular configuration file that allows for the replication of their results; they increase the maximum number of training iterations five-fold and set the threshold for ρ used in training higher by a factor of 2.25 relative to the default settings. Even so, if the random seed they used is changed their program can often crash or diverge (for both hard-coded and symbolic dynamics). This difference between the pendulum output and pendulum state feedback systems is due to combination of the fact that the output system (along with their 2D quadrotor state and output feedback systems) use a Lyapunov function of the quadratic form whereas the pendulum state feedback system used the neural network form, and that the system's equilibrium point is unstable.

We conducted a similar numerical integration experiment with the path tracking state feedback system (initially defined in (3), used in (4), elaborated on in Appendix section A.1.2) to evaluate our framework on a simple system with a stable equilibrium and a true neural network controller. We found that the controllers synthesized for that system always satisfy the Lyapunov constraints throughout the entire ROA approximation $\mathcal{S}$ (with the default training hyperparameters) as seen in Table 2. This result is also shown qualitatively in a representative 3D plot of the Lyapunov function and derivative in Figure 4 below by the fact that the entire derivative manifold lies below the $\Delta V = 0$ surface (black).

Metric	Author Hard-Coded System	Our Symbolic System
ρ	3.423426 ± 0.183006	3.329456 ± 0.102035
Fraction of $\mathcal{B}$ with $V(\xi) \leq \rho$	0.6072 ± 0.003	0.6103 ± 0.0031
Fraction of $\mathcal{B}$ with $\Delta V(\xi) \leq 0$	1 ± 0	1 ± 0
Fraction of $\mathcal{B}$ in Intersection	0.6072 ± 0.003	0.6103 ± 0.0031
Fraction of $\mathcal{S}$ where $\Delta V(\xi) > 0$	0 ± 0	0 ± 0

Table 2: Path Tracking State Feedback Metrics, Median $\pm$ Median Absolute Deviation, $N = 25$

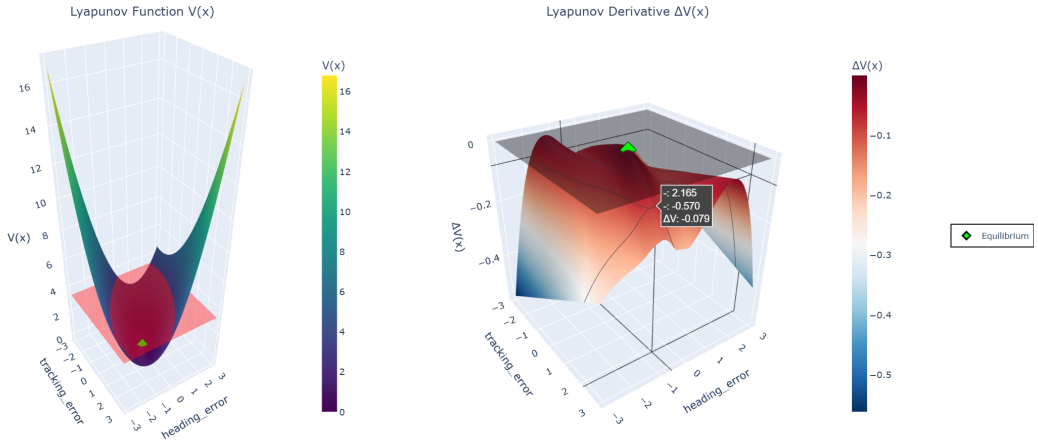


Figure 4: 3D Lyapunov and Lyapunov Derivative Surfaces for the Path-Tracking System

The ultimate goal of this endeavor was to enable people to construct Lyapunov functions, controllers, and observers using the code base for arbitrary nonlinear dynamical systems. Using the definitions in (3) as a starting point, we tried to write new example and configuration files for two of the dynamical systems that are written about in (4) but do not have corresponding example files: cartpole and planar vertical take-off and landing (PVTOL). While we were able to construct appropriate dynamical systems and we were able to start training controllers and Lyapunov functions, the programs did not converge within the allotted time for the GPU partition on the Northeastern University Research Computing Cluster (8 hours). We believe that this is due to the sensitivity of the algorithm for network training to various hyperparameters, mainly the threshold multiplier γ, Lyapunov derivative convergence term κ, and the set of candidate ROA states $\xi^{(i)}_{\text{candidate}}$ for L_{ROA}. For the candidate states in particular, the authors suggest in their Appendix to use a reference Lyapunov function (derived using the solutions to the continuous time or discrete time algebraic Riccati equation) but give no guidelines on how many candidate states should be considered.

6 Conclusion and Future Work

Our goal for this project was to enable people to use the code base developed to support (4) with arbitrary nonlinear dynamical systems, and we made considerable progress in that direction. We were able to successfully develop a user-friendly way to specify dynamical systems with SymPy, discretize them, and simulate them. We tested and verified that the framework we developed to define dynamical systems integrated with training scripts provided by the authors of (4) to synthesize Lyapunov functions, controllers, and observers that performed similarly to their hard-coded dynamical systems. Additionally, we added novel functionality to create Lyapunov function and Lyapunov derivative plots for qualitative evaluation of networks, along with numerical integration capabilities for quantitative comparisons. However, we were unable to truly extend the code base to construct Lyapunov functions, controllers, and observers for explicitly novel systems. We had to rely on existing configuration files and predetermined hyperparameter values to get our code to work with their code base.

The future work needed to realize a user-friendly framework for CLF, controller, and observer neural network synthesis is straightforward. First and foremost, immediate next steps should focus on allowing users to easily find and tune training algorithm hyperparameters given a dynamical system specification. Once the code base can truly construct Lyapunov candidates, controllers, and observers for any dynamical system that can be described with this framework, the next step will be to validate that the current framework is compatible with the verification algorithm code. As was demonstrated by the failure of some Lyapunov candidates to truly be control Lyapunov functions, the training algorithm is not guaranteed to output valid CLFs. The instructions for creating specifications to verify the Lyapunov, controller, and observer networks are somewhat unclear, so this will be a non-trivial task.

References

[1] Aleksandr Mikhailovich Lyapunov, "The general problem of the stability of motion," *International Journal of Control*, vol. 55, no. 3, pp. 531–534, 1892.

[2] C. Dawson, S. Gao, and C. Fan, "Safe Control With Learned Certificates: A Survey of Neural Lyapunov, Barrier, and Contraction Methods for Robotics and Control," *IEEE Transactions on Robotics*, vol. 39, pp. 1749–1767, June 2023.

[3] J. Wu, A. Clark, Y. Kantaros, and Y. Vorobeychik, "Neural Lyapunov Control for Discrete-Time Systems," in *Advances in Neural Information Processing Systems* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), vol. 36, pp. 2939–2955, Curran Associates, Inc., 2023.

[4] L. Yang, H. Dai, Z. Shi, C.-J. Hsieh, R. Tedrake, and H. Zhang, "Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation," in *Proceedings of the 41st International Conference on Machine Learning*, pp. 56033–56046, PMLR, July 2024. ISSN: 2640-3498.

[5] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, (Red Hook, NY, USA), pp. 4944–4953, Curran Associates Inc., Dec. 2018.

[6] K. Xu, Z. Shi, H. Zhang, Y. Wang, K.-W. Chang, M. Huang, B. Kailkhura, X. Lin, and C.-J. Hsieh, "Automatic perturbation analysis for scalable certified robustness and beyond," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, (Red Hook, NY, USA), pp. 1129–1141, Curran Associates Inc., Dec. 2020.

[7] K. Xu, H. Zhang, S. Wang, Y. Wang, S. Jana, X. Lin, and C.-J. Hsieh, "Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers," Mar. 2021. arXiv:2011.13824 [cs].

[8] H. Zhang, S. Wang, K. Xu, L. Li, B. Li, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "General Cutting Planes for Bound-Propagation-Based Neural Network Verification," Dec. 2022. arXiv:2208.05740 [cs].

[9] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Neural Network Robustness Verification," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 29909–29921, Curran Associates, Inc., 2021.

[10] Z. Shi, Q. Jin, Z. Kolter, S. Jana, C.-J. Hsieh, and H. Zhang, "Neural Network Verification with Branch-and-Bound for General Nonlinearities," Feb. 2025. arXiv:2405.21063 [cs].

[11] "Verified-Intelligence/Lyapunov_stable_nn_controllers," Nov. 2025. original-date: 2024-04-09T00:23:27Z.

[12] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "SymPy: symbolic computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. Publisher: PeerJ Inc.

[13] P. T. Inc, "Collaborative data science," 2015. Place: Montreal, QC Publisher: Plotly Technologies Inc.

[14] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, and P. van Mulbregt, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, Mar. 2020. Publisher: Nature Publishing Group.

[15] H. K. Khalil, "Lyapunov's Stability Theory," in *Encyclopedia of Systems and Control*, pp. 685–690, Springer, London, 2015.

[16] A. A. Ahmadi and A. Majumdar, "Some Applications of Polynomial Optimization in Operations Research and Real-Time Decision Making," Apr. 2015. arXiv:1504.06002 [math].

[17] Y.-C. Chang, N. Roohi, and S. Gao, "Neural Lyapunov Control," in *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019.

[18] R. Zhou, T. Quartz, H. D. Sterck, and J. Liu, "Neural Lyapunov Control of Unknown Nonlinear Systems with Stability Guarantees," Oct. 2022. arXiv:2206.01913 [eess].

[19] A. Abate, D. Ahmed, M. Giacobbe, and A. Peruffo, "Formal Synthesis of Lyapunov Neural Networks," *IEEE Control Systems Letters*, vol. 5, pp. 773–778, July 2021. arXiv:2003.08910 [eess].

[20] S. Chen, M. Fazlyab, M. Morari, G. J. Pappas, and V. M. Preciado, "Learning lyapunov functions for hybrid systems," in *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*, HSCC '21, (New York, NY, USA), pp. 1–11, Association for Computing Machinery, May 2021.

[21] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation," in *Proceedings of The 2nd Conference on Robot Learning*, pp. 651–673, PMLR, Oct. 2018. ISSN: 2640-3498.

[22] C. Dawson, Z. Qin, S. Gao, and C. Fan, "Safe Nonlinear Control Using Robust Neural Lyapunov-Barrier Functions," in *Proceedings of the 5th Conference on Robot Learning*, pp. 1724–1735, PMLR, Jan. 2022. ISSN: 2640-3498.

[23] J. Liu, Y. Meng, M. Fitzsimmons, and R. Zhou, "Towards Learning and Verifying Maximal Neural Lyapunov Functions," Apr. 2023. arXiv:2304.07215 [math].

[24] R. Sun, M. L. Greene, D. M. Le, Z. I. Bell, G. Chowdhary, and W. E. Dixon, "Lyapunov-Based Real-Time and Iterative Adjustment of Deep Neural Networks," *IEEE Control Systems Letters*, vol. 6, pp. 193–198, 2022.

[25] H. Dai, B. Landry, L. Yang, M. Pavone, and R. Tedrake, "Lyapunov-stable neural-network control," Sept. 2021. arXiv:2109.14152 [cs].

A Technical Appendices and Supplementary Material

A.1 System Definitions and Code Implementations

Docstrings (where most of the explanations regarding system parameters are) have been edited out for brevity, full code can be found at `https://github.com/gbenezer/Lyapunov_Stable_NN_Controllers_Custom_Dynamics/blob/main/neural_lyapunov_training/symbolic_systems.py`.

A.1.1 Inverted Pendulum

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix}\theta\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \dfrac{mgl\sin(\theta)+u-b\dot{\theta}}{ml^2}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \dfrac{-b}{ml^2}\cdot\dot{\theta}+\dfrac{g}{l}\cdot\sin(\theta)+\dfrac{1}{ml^2}\cdot u\end{bmatrix}; \quad h\left(\begin{bmatrix}\theta\\ \dot{\theta}\end{bmatrix}\right) = [\theta]$$

Code Definition:

```
class SymbolicPendulum(SymbolicDynamicalSystem):

    def __init__(
        self, m: float = 1.0,
        l: float = 1.0,
        beta: float = 1.0,
        g: float = 9.81
    ):
        super().__init__()
        self.order = 1
        self.define_system(m, l, beta, g)

    def define_system(self, m_val, l_val, beta_val, g_val):
        theta, theta_dot = sp.symbols("theta theta_dot",
            real=True)
        u = sp.symbols("u",
            real=True)
        m, l, beta, g = sp.symbols("m l beta g",
            real=True,
            positive=True)

        self.parameters = {m: m_val,
                           l: l_val,
                           beta: beta_val,
                           g: g_val}
        self.state_vars = [theta,
                           theta_dot]
        self.control_vars = [u]
        self.output_vars = [theta]

        ml2 = m * l * l
        self._f_sym = sp.Matrix(
            [theta_dot,
            ((-beta / ml2) * theta_dot +
                (g / l) * sp.sin(theta) + u / ml2)]
        )
        self._h_sym = sp.Matrix([theta])
```

A.1.2 Path-Tracking System

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix} d_e \\ \theta_e \end{bmatrix} = \begin{bmatrix} \dot{d}_e \\ \dot{\theta}_e \end{bmatrix} = \begin{bmatrix} v \cdot \sin(\theta_e) \\ \frac{v \cdot \delta}{L} - \frac{\cos(\theta_e)}{\frac{R}{v} - \sin(\theta_e)} \end{bmatrix}; \quad h\left(\begin{bmatrix} d_e \\ \theta_e \\ \dot{d}_e \\ \dot{\theta}_e \end{bmatrix}\right) = \begin{bmatrix} d_e \\ \theta_e \end{bmatrix}$$

Code Definition:

```
class PathTracking(SymbolicDynamicalSystem):
    def __init__(self,
                 speed: float = 1.0,
                 length: float = 1.0,
                 radius: float = 10.0):
        super().__init__()
        self.order = 1
        self.speed_val = speed
        self.length_val = length
        self.radius_val = radius
        self.define_system(speed, length, radius)

    def define_system(self, speed_val, length_val, radius_val):
        d_e, theta_e = sp.symbols("d_e theta_e", real=True)
        delta = sp.symbols("delta", real=True)
        v, L, R = sp.symbols("v L R", real=True, positive=True)

        self.parameters = {v: speed_val, L: length_val, R: radius_val}

        self.state_vars = [d_e, theta_e]
        self.control_vars = [delta]
        self.output_vars = [d_e, theta_e]

        # Error dynamics
        sin_theta_e = sp.sin(theta_e)
        cos_theta_e = sp.cos(theta_e)

        # Lateral error rate
        d_e_dot = v * sin_theta_e

        # Heading error rate
        coef = R / v
        theta_e_dot = (v * delta / L) - (cos_theta_e / (coef - sin_theta_e))

        self._f_sym = sp.Matrix([d_e_dot, theta_e_dot])
        self._h_sym = sp.Matrix([d_e, theta_e])

    @property
    def x_equilibrium(self) -> torch.Tensor:
        return torch.zeros(2)

    @property
    def u_equilibrium(self) -> torch.Tensor:
        return torch.tensor([self.length_val / self.radius_val])
```

A.1.3 Quadrotor State Observation System

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\ddot{x}\\ \ddot{y}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix}\frac{-(u_1+u_2)\cdot\sin(\theta)}{m}\\ \frac{(u_1+u_2)\cdot\cos(\theta)}{m} - g\\ \frac{L\cdot(u_1-u_2)}{I}\end{bmatrix}; \quad h\left(\begin{bmatrix}x\\ y\\ \theta\\ \dot{x}\\ \dot{y}\\ \dot{\theta}\end{bmatrix}\right) = \begin{bmatrix}x\\ y\\ \theta\end{bmatrix}$$

Code Definition:

```
class SymbolicQuadrotor2D(SymbolicDynamicalSystem):
    def __init__(
        self,
        length: float = 0.25,
        mass: float = 0.486,
        inertia: float = 0.00383,
        gravity: float = 9.81,
    ):
        super().__init__()
        self.order = 2
        # Store values for backward compatibility
        self.length_val = length
        self.mass_val = mass
        self.inertia_val = inertia
        self.gravity_val = gravity
        self.define_system(length, mass, inertia, gravity)

    def define_system(self,
                      length_val,
                      mass_val,
                      inertia_val,
                      gravity_val):
        x, y, theta, x_dot, y_dot, theta_dot = sp.symbols(
            "x y theta x_dot y_dot theta_dot", real=True
        )
        u1, u2 = sp.symbols("u1 u2", real=True)
        L, m, I, g = sp.symbols("L m I g", real=True, positive=True)

        self.parameters = {L: length_val,
                           m: mass_val,
                           I: inertia_val,
                           g: gravity_val}
        self.state_vars = [x, y, theta, x_dot, y_dot, theta_dot]
        self.control_vars = [u1, u2]
        self.output_vars = [x, y, theta]

        # For second-order system, forward() returns acceleration
        dx_dot = (-1 / m) * sp.sin(theta) * (u1 + u2)
        dy_dot = (1 / m) * sp.cos(theta) * (u1 + u2) - g
        dtheta_dot = (L / I) * (u1 - u2)

        self._f_sym = sp.Matrix([dx_dot, dy_dot, dtheta_dot])
        self._h_sym = sp.Matrix([x, y, theta])

    @property
    def u_equilibrium(self) -> torch.Tensor:
        mg = self.mass_val * self.gravity_val
        return torch.tensor([mg / 2, mg / 2])
```

```
@property
def length(self):
    """For backward compatibility"""
    return self.length_val

@property
def mass(self):
    """For backward compatibility"""
    return self.mass_val

@property
def inertia(self):
    """For backward compatibility"""
    return self.inertia_val

@property
def gravity(self):
    """For backward compatibility"""
    return self.gravity_val
```

A.1.4 Quadrotor LIDAR System

State-Space Formulation

$$\frac{d}{dt}\begin{bmatrix}\dot{y}\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\ddot{y}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix}\frac{(u_1+u_2)\cdot\cos(\theta)}{m} - g - b\cdot\dot{y}\\ \frac{L\cdot(u_1-u_2)}{I} - b\cdot\dot{\theta}\end{bmatrix}; \quad h\left(\begin{bmatrix}y\\ \theta\\ \dot{y}\\ \dot{\theta}\end{bmatrix}\right) = \begin{bmatrix}d_{\theta_1}\\ d_{\theta_2}\\ d_{\theta_3}\\ d_{\theta_4}\end{bmatrix}$$

Code Definition:

```
class SymbolicQuadrotor2DLidar(SymbolicDynamicalSystem):
    def __init__(
        self,
        length: float = 0.25,
        mass: float = 0.486,
        inertia: float = 0.00383,
        gravity: float = 9.81,
        b: float = 0.0,
        H: float = 5.0,
        angle_max: float = 0.149 * np.pi,
        origin_height: float = 1.0,
    ):
        super().__init__()
        self.order = 2
        # Store values for backward compatibility
        self.length_val = length
        self.mass_val = mass
        self.inertia_val = inertia
        self.gravity_val = gravity
        self.b_val = b
        self.H = H
        self.angle_max = angle_max
        self.origin_height = origin_height
        self.define_system(
            length,
            mass,
            inertia,
            gravity,
            b,
            H,
            angle_max,
            origin_height
        )

    def define_system(
        self,
        length_val,
        mass_val,
        inertia_val,
        gravity_val,
        b_val,
        H_val,
        angle_max_val,
        origin_height_val,
    ):
        y, theta, y_dot, theta_dot = sp.symbols(
            "y theta y_dot theta_dot",
            real=True)
        u1, u2 = sp.symbols("u1 u2", real=True)
        L, m, I, g, b = sp.symbols("L m I g b", real=True, positive=True)
```

```
        self.parameters = {
            L: length_val,
            m: mass_val,
            I: inertia_val,
            g: gravity_val,
            b: b_val,
        }
        self.state_vars = [y, theta, y_dot, theta_dot]  # nx = 4
        self.control_vars = [u1, u2]

        # Dynamics
        dy_dot = (1 / m) * sp.cos(theta) * (u1 + u2) - g - b * y_dot
        dtheta_dot = (L / I) * (u1 - u2) - b * theta_dot

        self._f_sym = sp.Matrix([dy_dot, dtheta_dot])

        # Lidar observation model
        # Create 4 lidar rays at different angles
        ny = 4
        lidar_rays = []

        for i in range(ny):
            # Linearly spaced angles from -angle_max to +angle_max
            angle_offset = (-angle_max_val +
                            i * (2 * angle_max_val / (ny - 1)))
            phi = theta - angle_offset

            # Basic ray calculation: distance = (height) / cos(angle)
            ray_distance = (y + origin_height_val) / sp.cos(phi)

            # Smooth approximation of clamping to [0, H]
            # Use smooth functions to maintain differentiability
            # smooth_clamp(x, 0, H) approx max(0, min(x, H))

            # Soft ReLU for lower bound, soft minimum for upper bound

            eps = 1e-6  # Small constant for numerical stability

            # Soft ReLU: max(0, x) approx (x + sqrt(x^2 + eps))/2
            soft_relu = (ray_distance +
                         sp.sqrt(ray_distance**2 + eps)) / 2

            # Soft min(x, H): H - soft_relu(H - x)
            clamped_ray = (
                H_val
                - (H_val - soft_relu +
                   sp.sqrt((H_val - soft_relu) ** 2 + eps)) / 2
            )

            lidar_rays.append(clamped_ray)

        self._h_sym = sp.Matrix(lidar_rays)
        self.output_vars = [
            sp.Symbol(f"lidar_{i}", real=True) for i in range(ny)
        ]  # ny = 4

    @property
    def u_equilibrium(self) -> torch.Tensor:
```

```
        mg = self.mass_val * self.gravity_val
        return torch.tensor([mg / 2, mg / 2])

    @property
    def length(self):
        """For backward compatibility"""
        return self.length_val

    @property
    def mass(self):
        """For backward compatibility"""
        return self.mass_val

    @property
    def inertia(self):
        """For backward compatibility"""
        return self.inertia_val

    @property
    def gravity(self):
        """For backward compatibility"""
        return self.gravity_val

    @property
    def b(self):
        """For backward compatibility"""
        return self.b_val
```

A.1.5 Cartpole System

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix}\dot{x}\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\ddot{x}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix} \dfrac{F - b\cdot\dot{x} + m_{pole}\cdot\sin(\theta)\cdot(l\cdot\theta^2 - g\cdot\cos(\theta))}{m_{cart} + m_{pole}\cdot\sin^2(\theta)} \\ \dfrac{(F - b\cdot\dot{x} + m_{pole}\cdot l\cdot\theta^2\cdot\sin(\theta))\cdot\cos(\theta) - (m_{cart}+m_{pole})\cdot g\cdot\sin(\theta)}{l\cdot(m_{cart} + m_{pole}\cdot\sin^2(\theta))} \end{bmatrix}$$

$$h\left(\begin{bmatrix}x\\ \theta\\ \dot{x}\\ \dot{\theta}\end{bmatrix}\right) = \begin{bmatrix}x\\ \theta\end{bmatrix}$$

Code Definition:

```
class CartPole(SymbolicDynamicalSystem):
    def __init__(
        self,
        m_cart: float = 1.0,
        m_pole: float = 0.1,
        length: float = 0.5,
        gravity: float = 9.81,
        friction: float = 0.1,
    ):
        super().__init__()
        self.order = 2
        # Store values
        self.m_cart_val = m_cart
        self.m_pole_val = m_pole
        self.length_val = length
        self.gravity_val = gravity
        self.friction_val = friction
        self.define_system(m_cart,
                           m_pole,
                           length,
                           gravity,
                           friction)

    def define_system(
        self,
        m_cart_val,
        m_pole_val,
        length_val,
        gravity_val,
        friction_val
    ):
        # State variables
        x, theta, x_dot, theta_dot = sp.symbols("x theta x_dot theta_dot",
            real=True)
        F = sp.symbols("F", real=True)

        # Parameters
        mc, mp, l, g, b = sp.symbols("mc mp l g b",
                                     real=True,
                                     positive=True)

        self.parameters = {
            mc: m_cart_val,
            mp: m_pole_val,
```

```
            l: length_val,
            g: gravity_val,
            b: friction_val,
        }

        self.state_vars = [x, theta, x_dot, theta_dot]
        self.control_vars = [F]
        self.output_vars = [x, theta]

        # Dynamics (derived from Euler-Lagrange equations)
        # Total mass
        M = mc + mp

        # Sin and cos of theta
        sin_theta = sp.sin(theta)
        cos_theta = sp.cos(theta)

        # Denominator for both equations
        denom = M - mp * cos_theta**2

        # Cart acceleration
        x_ddot = (
            F
            - b * x_dot
            + mp * l * theta_dot**2 * sin_theta
            - mp * g * sin_theta * cos_theta
        ) / denom

        # Pole angular acceleration
        theta_ddot = (
            F * cos_theta
            - b * x_dot * cos_theta
            + mp * l * theta_dot**2 * sin_theta * cos_theta
            - M * g * sin_theta
        ) / (l * denom)

        # Second-order system: forward() returns accelerations
        self._f_sym = sp.Matrix([x_ddot, theta_ddot])
        self._h_sym = sp.Matrix([x, theta])

    @property
    def x_equilibrium(self) -> torch.Tensor:
        """Upright equilibrium at origin"""
        return torch.zeros(4)

    @property
    def u_equilibrium(self) -> torch.Tensor:
        """No force needed at equilibrium"""
        return torch.zeros(1)
```

A.1.6 PVTOL System

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\ddot{x}\\ \ddot{y}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{y}\cdot\dot{\theta} - g\cdot\sin(\theta)\\ \frac{u_1+u_2}{m} - \dot{x}\cdot\dot{\theta} - g\cdot\cos(\theta)\\ \frac{d\cdot(u_1-u_2)}{I}\end{bmatrix}; \quad h\left(\begin{bmatrix}x\\ y\\ \theta\\ \dot{x}\\ \dot{y}\\ \dot{\theta}\end{bmatrix}\right) = \begin{bmatrix}x\\ y\\ \theta\end{bmatrix}$$

Code Definition:

```
class PVTOL(SymbolicDynamicalSystem):
    def __init__(
        self,
        length: float = 0.25,
        mass: float = 4.0,
        inertia: float = 0.0475,
        gravity: float = 9.8,
        dist: float = 0.25,
    ):
        super().__init__()
        self.order = 2
        # Store values
        self.length_val = length
        self.mass_val = mass
        self.inertia_val = inertia
        self.gravity_val = gravity
        self.dist_val = dist
        self.define_system(length, mass, inertia, gravity, dist)

    def define_system(self,
                      length_val,
                      mass_val,
                      inertia_val,
                      gravity_val,
                      dist_val):
        # State variables (position and velocity in body frame)
        x, y, theta, x_dot, y_dot, theta_dot = sp.symbols(
            "x y theta x_dot y_dot theta_dot", real=True
        )
        u1, u2 = sp.symbols("u1 u2", real=True)

        # Parameters
        L, m, I, g, d = sp.symbols("L m I g d",
                                   real=True,
                                   positive=True)

        self.parameters = {
            L: length_val,
            m: mass_val,
            I: inertia_val,
            g: gravity_val,
            d: dist_val,
        }

        self.state_vars = [x, y, theta, x_dot, y_dot, theta_dot]
        self.control_vars = [u1, u2]
        self.output_vars = [x, y, theta]
```

```
        # Rotation from body to world frame
        sin_theta = sp.sin(theta)
        cos_theta = sp.cos(theta)

        # Acceleration dynamics in body frame
        x_ddot = y_dot * theta_dot - g * sin_theta
        y_ddot = -x_dot * theta_dot - g * cos_theta + (u1 + u2) / m
        theta_ddot = (u1 - u2) * d / I

        # For second-order system, forward() returns accelerations
        self._f_sym = sp.Matrix([x_ddot, y_ddot, theta_ddot])
        self._h_sym = sp.Matrix([x, y, theta])

    @property
    def x_equilibrium(self) -> torch.Tensor:
        return torch.zeros(6)

    @property
    def u_equilibrium(self) -> torch.Tensor:
        return torch.full((2,), self.mass_val * self.gravity_val / 2)

    @property
    def length(self):
        return self.length_val

    @property
    def mass(self):
        return self.mass_val

    @property
    def inertia(self):
        return self.inertia_val

    @property
    def gravity(self):
        return self.gravity_val

    @property
    def dist(self):
        return self.dist_val
```